POViewer

Point of View Viewer

Phase 2 Design Specification

v2.0 — BKC AI Triad Project — 2026

Phase 1 — COMPLETE	Phase 2 — THIS DOCUMENT
Static prototype • Hardcoded data • Three-pane layout • Taxonomy loading • Highlight system • Filter toggles • Pane 3 detail card	Live AI • PDF rendering • Annotations • Cross-source view • Taxonomy Gap view • Export/sharing • Long documents • Public release

1. Phase 2 Overview

Phase 1 proved the POViewer interaction model: the three-pane layout, the highlight system, POV filter toggles, and the Pane 3 detail card all validated with the BKC team using hardcoded data. Phase 2 connects that proven UI to real infrastructure. Every capability deferred from Phase 1 is addressed here.

Phase 2 has seven parallel workstreams. They are largely independent and can be staffed and shipped in any order, though the AI Analysis Engine (Section 3) is a prerequisite for the Cross-Source Aggregation View (Section 6).

#	Workstream	Phase 1 state	Phase 2 delivers
3	AI Analysis Engine	Hardcoded data	Live Gemini/Claude calls, real point segmentation
4	PDF Rendering	Not built	Full in-app PDF viewer with highlight overlay
5	User Annotations	Not built	Override, add, remove, and note AI mappings
6	Cross-Source Aggregation	Not built	Taxonomy-node-centric view across all sources
7	Taxonomy Gap View	Not built	First-class view of all unmapped points
8	Export & Sharing	Not built	Notebook bundles, report export, citation copy
9	Long Document Handling	Not built	Chunked analysis, progressive rendering
10	Public Release Readiness	Internal BKC only	Onboarding, API key UX, telemetry, accessibility

Attribute	Decision
Platform	Desktop (Electron) — unchanged from Phase 1. Web version is Phase 3.
AI provider	User-supplied API key via UI settings. Supports Gemini and Claude. Key stored in OS keychain.
Taxonomy model	Unchanged: taxonomy-agnostic, user loads JSON files, paths persisted across sessions.
Public release gate	Phase 2 ships to BKC team + selected external beta users. Full public release is Phase 3.

2. Phase 1 Open Questions — Resolutions

The Phase 1 spec listed six open questions for Phase 2. Each is resolved here before the detailed workstream sections.

Phase 1 question	Phase 2 resolution
Multi-source aggregation view: can Pane 3 show mappings ACROSS sources, not just within one?	Yes — a new fourth view mode in Pane 3 called the Node View. See Section 6.
Can a user manually override an AI mapping — add, remove, or correct a highlight?	Yes — full annotation layer on top of AI output. Annotations are versioned separately from AI results. See Section 5.
Should Notebooks be shareable / exportable as a bundle?	Yes — .poviewer bundle format (ZIP). Includes sources, snapshots, analysis, annotations, and taxonomy snapshots. See Section 8.
How does the tool handle very long documents (100+ pages)?	Chunked analysis with a progress indicator. Rendering is virtual-scrolled. See Section 9.
Should Pane 3 include a Taxonomy Gap view listing all UNMAPPED points across all sources?	Yes — a dedicated Gap View tab in Pane 3 surfaces all unmapped points sorted by frequency. See Section 7.
Web-hosted version vs. desktop only for public release?	Desktop only for Phase 2. Web version is scoped to Phase 3 to avoid backend/auth complexity while the tool matures.

3. Workstream A — Live AI Analysis Engine

This workstream replaces the hardcoded analysis data with real AI calls. It is the prerequisite for every other Phase 2 feature and should be the first workstream completed.

3.1 API Provider Configuration

Users configure their AI provider in Settings → AI Provider. The settings panel accepts:

Field	Details
Provider	Dropdown: Gemini (Google) | Claude (Anthropic). Extensible for future providers.
API Key	Password-masked text field. On save, key is written to the OS keychain via Electron's safeStorage API — never to disk in plaintext.
Model	Dropdown filtered by provider. Gemini: gemini-1.5-flash (default), gemini-1.5-pro, gemini-2.0-flash-exp. Claude: claude-3-5-haiku (default), claude-3-5-sonnet, claude-3-opus.
Test button	Sends a minimal test prompt (“Respond with: OK”) and shows latency + model confirmation.
Token budget	Optional monthly token cap with a usage counter. Warn at 80%, block at 100%.

> **Key security model**
> The API key is stored using Electron's safeStorage.encryptString() on Mac (Keychain) and Windows (DPAPI). It is never written to notebook.json or any file in the Notebook folder. If a Notebook bundle is exported and shared, no key is included.

3.2 Analysis Pipeline — Two-Stage Architecture

Analysis runs in two sequential stages per source document. The stages are decoupled so Stage 1 can be re-run cheaply when the taxonomy changes without re-running Stage 2.

Stage 1: Segmentation

The AI reads the full document and returns a flat list of Points — each being a distinct logical claim with a character-offset range into snapshot.md. This stage does NOT perform taxonomy mapping; it only identifies claim boundaries.

Input	Output
snapshot.md (full text)	Array of Point objects: { point_id, text, char_start, char_end }

Segmentation is stable: the same document produces the same Point list regardless of which taxonomies are active. This means segmentation only needs to re-run when the source document changes, not when taxonomies change.

Stage 2: Mapping

The AI maps each Point against the active taxonomy set. This stage IS taxonomy-dependent and re-runs when the taxonomy version changes. The mapping prompt sends the full taxonomy JSON + one Point at a time (or in batches of 5–10 for efficiency).

Input	Output per Point
Point text + taxonomy JSON (all active POV files)	mappings[], unmapped bool, unmapped_hint, cross_cutting_collision flag

Why split segmentation from mapping?
A taxonomy update (the common case during active research) only requires re-running Stage 2 — which is faster and cheaper because the Point boundaries are already known. Re-segmentation is only needed when the source document itself is updated. This design saves significant API cost during the iterative taxonomy-building phase of the project.

3.3 Analysis Trigger Logic

Event	What re-runs
Source first selected	Stage 1 (segmentation) + Stage 2 (mapping)
Taxonomy file changed + version bumped	Stage 2 only (mapping) — for all sources tagged with changed POV
User clicks ‘Re-analyse’ on a source	Stage 2 only (mapping) — unless source snapshot has changed
Source snapshot updated (URL re-fetched)	Stage 1 + Stage 2
New taxonomy loaded mid-session	Stage 2 only — for all sources, using new taxonomy

3.4 Analysis Status Model

Status	Meaning	Visual
not_analysed	Source has never been analysed	Gray dot
queued	Waiting in the local job queue	Clock icon
segmenting	Stage 1 in progress	Spinner: ‘Segmenting...’
mapping	Stage 2 in progress, shows % complete	Progress bar

Status	Meaning	Visual
complete	Analysis current for active taxonomy version	Green dot
stale	Taxonomy version changed since last analysis	Orange ‘Stale’ badge
partial	Stage 1 complete, Stage 2 in progress or interrupted	Half-filled circle
error	API call failed; error details in tooltip	Red ‘Error’ badge
rate_limited	API returned 429; will auto-retry with exponential backoff	Amber ‘Waiting...’

3.5 Concurrency and Queue

Analysis runs in a local FIFO queue managed by the Electron main process. The queue:

- Processes one source at a time by default (configurable to 2 concurrent in Settings for fast models).
- Persists across app restarts — incomplete jobs resume from the last completed Point batch.
- Shows a global progress indicator in the toolbar when any job is active.
- Implements exponential backoff (2s, 4s, 8s, 16s, max 60s) for 429 rate-limit responses.
- Respects the user’s token budget — pauses the queue when the monthly cap is reached.

3.6 Prompt Architecture

Both prompts are stored in the app as editable templates (advanced settings) so power users can tune them without code changes. The templates use Handlebars-style {{variable}} substitution.

Prompt	Variables	Temperature	Max tokens
Segmentation prompt	{{document_text}}	0.1	4096
Mapping prompt	{{point_text}}, {{taxonomy_json}}, {{output_schema}}	0.1	1024 per point

4. Workstream B — PDF Rendering

Phase 1 rendered only Markdown snapshots. Phase 2 adds native PDF display in Pane 2 so users can read the original document layout — charts, tables, page numbers — while still seeing the AI highlights. This matters especially for academic papers and policy reports where layout carries meaning.

4.1 Rendering Architecture

PDF rendering uses PDF.js (pdfjs-dist) embedded in the Electron renderer process. The PDF is loaded from the raw/ folder in the Notebook directory, not re-fetched from the network.

Layer	Technology	Responsibility
PDF decode	pdfjs-dist (Mozilla)	Decode PDF binary, render pages to canvas elements
Text layer	PDF.js TextLayer	Invisible text overlay on each canvas for selection + highlight anchoring
Highlight layer	Custom SVG overlay	Coloured rectangles positioned over TextLayer spans, one per Point
Scroll engine	Virtual scroll (custom)	Only render pages within ±2 pages of viewport to handle 100+ page docs

4.2 Highlight Anchoring in PDF Mode

Highlights in PDF mode use character offsets into the PDF's extracted text stream (which maps to the same text used in snapshot.md). The TextLayer exposes each character's bounding box, allowing precise rectangle placement. When a Point spans multiple lines or columns, multiple rectangles are drawn and visually connected with a thin colored rule.

> **Anchoring resilience**
> PDF text extraction is not always perfectly consistent between runs (ligatures, hyphenation, column order). The highlight anchor stores both the character offset AND a short text fingerprint (the first 40 characters of the Point). On load, it attempts offset-first, falling back to fuzzy text search if the offset lands on a different string than the fingerprint. This handles re-ingested PDFs without losing all annotations.

4.3 View Toggle

A toggle in the Pane 2 toolbar switches between two views for sources that have both a PDF and a snapshot:

Mode	What it shows	When to use
PDF view	Native PDF rendered with highlight overlay	Reading experience; fidelity to original layout
Snapshot view	Markdown snapshot rendered as clean HTML	Faster; better for copy-paste; works when PDF unavailable

URL-sourced documents that are not PDFs always open in Snapshot view. PDF-sourced documents open in PDF view by default; users can switch to Snapshot view at any time.

5. Workstream C — User Annotations

The Phase 1 spec identified this as essential for research trust: analysts must be able to say ‘the AI got this wrong’. Phase 2 implements a full annotation layer that sits on top of AI output without modifying the AI’s stored results. AI output and human corrections are always distinguishable.

5.1 Design Principle: Non-Destructive Override

User annotations never overwrite AI analysis data. The AI’s original output is preserved in analysis/<doc-id>.json unchanged. Annotations are stored separately in annotations/<doc-id>.json. The rendering layer merges both at display time, with annotations taking visual precedence.

> **Why non-destructive?**
> Research integrity requires knowing what the AI said vs. what a human corrected. When the taxonomy changes and the AI re-runs, the human annotations are not blown away — they persist and are reviewed against the new AI output. A diff mode shows where AI and human annotations now agree vs. still diverge.

5.2 Annotation Types

Type	What it does	How triggered
Override mapping	Replace an AI-assigned taxonomy node with a different node	Right-click highlight → ‘Change node’ → node picker
Override alignment	Flip AGREES ↔ CONTRADICTS on an existing mapping	Right-click highlight → ‘Flip alignment’
Add mapping	Add a new mapping to a Point that the AI didn’t find	Right-click highlight → ‘Add mapping’ → node picker
Remove mapping	Remove an AI mapping the analyst disagrees with	Pane 3 detail card → mapping row → trash icon
Add new point	Create a Point on text the AI didn’t segment	Select text in Pane 2 → ‘Create Point’ in context menu
Suppress point	Hide a Point entirely (AI segmentation error — not a claim)	Pane 3 detail card → ‘Suppress Point’
Research note	Attach a free-text note to any Point (not a taxonomy change)	Pane 3 detail card → ‘Add note’ icon

5.3 Visual Differentiation: AI vs. Human

Source	Visual indicator in Pane 2	Visual indicator in Pane 3
AI-only mapping	Solid filled highlight	No badge

Source	Visual indicator in Pane 2	Visual indicator in Pane 3
Human-overridden	Solid fill + pencil icon ✏	'Edited' badge on affected mapping row
Human-added mapping	Solid fill + plus icon ✚	'Human' badge on mapping row
Human-suppressed	Strikethrough / greyed-out text span	Shows in Suppressed tab in Pane 3
Has research note	Small comment icon 💬 at end	Note card shown below mapping blocks

5.4 Annotation Storage Schema

Each annotation file mirrors the Point structure:

Field	Type	Description
doc_id	string	Parent document
annotation_id	string	Stable ID, e.g. 'ann-012'
point_id	string	The Point this annotation targets
type	string	override_mapping | override_alignment | add_mapping | remove_mapping | add_point | suppress | note
target_node_id	string?	For override/add: the replacement or new node ID
alignment	string?	For override_alignment: 'agrees' or 'contradicts'
note_text	string?	For note type: free-text content
created_by	string	Username or 'local' (used when Notebook is shared)
created_at	string	ISO timestamp
superseded_by	string?	ID of a later annotation that replaces this one (audit trail)

5.5 Re-analysis Behaviour

When the AI re-runs analysis on a source (e.g., after a taxonomy version bump), the following merge logic applies:

- Human annotations for Points that still exist and whose text fingerprints still match: preserved unchanged.
- Human annotations for Points that no longer exist (AI re-segmented the document differently): moved to a Detached Annotations review list where the analyst can re-apply or discard them.
- A diff badge appears on the source in Pane 1: '12 AI changes, 3 annotations need review.'

6. Workstream D — Cross-Source Aggregation View

Phase 1 answered the question ‘how does this document treat this taxonomy node?’ Phase 2 answers the research question that matters more: ‘how do ALL documents in this Notebook treat this taxonomy node?’ This is the view that makes POViewer a research instrument rather than a document reader.

6.1 Node View — New Pane 3 Mode

Pane 3 gains a fourth mode alongside the existing Detail Card and Filter Toggle modes: the Node View. It is activated by clicking a taxonomy node ID anywhere in the UI, or via the Node View tab at the top of Pane 3.

The Node View shows, for a selected taxonomy node, every Point across every source in the Notebook that maps to it — along with whether each Point agrees or contradicts. This is the full cross-source picture for that concept.

6.1.1 Node View Layout

Section	Content
Node header	Node ID, label, description, POV camp color chip, category badge (Goals/Values / Data/Facts / Methods)
Stance summary	Horizontal bar chart: how many Points agree vs. contradict this node, across all selected sources. Color-coded by source.
Source breakdown	One collapsible block per source. Shows: source title, overall stance toward this node, count of Points, list of Point excerpts with agree/contradict icons.
Jump to source	Click any Point excerpt → Pane 2 jumps to that source and scrolls to that Point, highlighting it.
Cross-node links	Other taxonomy nodes that frequently co-occur with this one across Points. Clicking navigates to that node’s Node View.

6.2 Cross-Source Conflict Detection

When the Node View is open for a Data/Facts node, it surfaces cross-source conflicts: cases where Source A’s Point AGREES with the node while Source B’s Point CONTRADICTS the same node. These are the empirical disagreements the AI Triad taxonomy is designed to track.

Conflict example

Node acc-data-001 (Compute-to-intelligence scaling continues predictably): Altman 2024 (AGREES — 3 Points) vs. LeCun 2024 (CONTRADICTS — 2 Points) vs. Mitchell 2023 (NEUTRAL — 1 Point). The Node View makes this dispute visible at a glance across all three documents.

6.3 Notebook-Level Stance Heatmap

A new Notebook Overview screen (accessible from the Notebook bar) shows a heatmap: rows are taxonomy nodes, columns are sources, cells show the stance of that source toward that node (colour intensity = strength, icon = agrees/contradicts/neutral/absent). This gives the research team a one-screen summary of where all the documents in a Notebook agree, disagree, and stay silent.

Cell state	Color	Meaning
Strongly agrees	Deep camp color	Multiple HIGH-strength agreeing Points
Agrees	Medium camp color	At least one agreeing Point
Neutral	Light gray	Points exist but stance is neutral
Contradicts	Camp color + diagonal line	At least one contradicting Point
Silent	White / empty	No Points mapped to this node in this source

7. Workstream E — Taxonomy Gap View

Every unmapped Point is a potential signal that the taxonomy needs to grow. In Phase 1, unmapped Points were surfaced in individual Pane 3 detail cards but were invisible in aggregate. Phase 2 promotes taxonomy gaps to a first-class view.

7.1 Gap View Tab

A ‘Gaps’ tab appears in Pane 3 alongside the existing Filter Toggles and Detail Card modes. The Gap View is always available regardless of what is selected in Pane 2.

7.1.1 Gap View Layout

Section	Content
Summary bar	Total unmapped Points across all selected sources. Breakdown by Gemini’s suggested POV camp.
Cluster list	Unmapped Points grouped by semantic similarity. Gemini clusters them during analysis into candidate concepts (e.g., ‘regulatory sandbox’, ‘dual-use research’). Each cluster shows a suggested label and the Points that belong to it.
Frequency sort	Clusters sorted by frequency (most Points first). High-frequency gaps are the strongest candidates for new taxonomy nodes.
Promote button	For each cluster: a ‘Propose Node’ button opens the Taxonomy Propose flow (Section 7.2).
Source filter	Filter gaps to a specific source, or show across all selected sources.

7.2 Taxonomy Propose Flow

When an analyst clicks ‘Propose Node’ on a gap cluster, a side panel opens with a pre-filled form:

- Suggested node ID (auto-generated following the existing convention: [camp]-[category]-[NNN])
- POV camp selector (pre-filled from Gemini’s suggestion)
- Category selector: Goals/Values | Data/Facts | Methods
- Label field (pre-filled from the cluster label, editable)
- Description field (pre-filled from the cluster Points, editable)
- Evidence quotes (the specific Point texts that motivated this proposal, linked to their sources)

Clicking ‘Save Proposal’ writes a proposal entry to taxonomy-proposals/<node-id>.json. These proposals are NOT immediately added to the active taxonomy — they require human review and a deliberate edit of the taxonomy JSON file + version bump. The Gap View shows a ‘Proposed’ badge on clusters with pending proposals.

Design intent

The Propose flow is intentionally lightweight and non-committal. It collects structured evidence for taxonomy expansion without modifying the taxonomy itself. This keeps taxonomy changes as deliberate human decisions, not automated mutations. The proposals file is designed to be the agenda for the research team's taxonomy working group sessions.

7.3 Cross-Cutting Promotion Detector

A background analysis pass (runs after all sources in a Notebook have been analysed) detects potential cross-cutting promotions: taxonomy nodes that are currently in one camp's file but are being cited by Points that also map to other camps' nodes. These candidates are surfaced in the Gap View under a 'Possible Cross-Cutting Concepts' section with a 'Promote to cross-cutting.json' button.

8. Workstream F — Export & Sharing

Phase 2 adds three export capabilities: Notebook bundles for sharing entire research packages, Markdown reports for publication-ready summaries, and citation copy for individual Points.

8.1 Notebook Bundle (.poviewer)

A .poviewer file is a ZIP archive containing a complete, self-contained snapshot of a Notebook. It can be opened by any POViewer installation without the receiving researcher needing to re-fetch or re-analyse any documents.

Bundle contains	Notes
notebook.json	Name, description, metadata. API key NOT included.
sources/<id>/	snapshot.md, metadata.json, raw/ for all sources
analysis/<id>.json	Full AI analysis for all analysed sources
annotations/<id>.json	All human annotations (with created_by field)
taxonomy-snapshots/	Frozen copies of all taxonomy JSON files at export time
MANIFEST.json	Bundle version, export timestamp, source app version, taxonomy version

When a .poviewer bundle is opened on another machine, POViewer loads from the frozen taxonomy snapshots inside the bundle. The receiving researcher can optionally switch to their locally loaded taxonomy files if they have a newer version.

> **No API key in bundles**
> The API key is never included in exports. If the receiving researcher wants to re-analyse sources or run the Gap View's background pass, they need their own API key configured in their local Settings. Shared bundles are read-only by default — annotations can be added locally but are not pushed back to the sender.

8.2 Markdown Report Export

File → Export → Notebook Report generates a structured Markdown file covering the entire Notebook. Sections are configurable (checkboxes before export):

- Executive summary: Notebook name, source count, analysis date, dominant POV stances
- Per-source summaries: title, one-liner, overall stance scores table, top 5 Points per camp
- Cross-source conflict log: all factual conflicts detected across sources for Data/Facts nodes
- Taxonomy Gap report: all proposed nodes with supporting evidence
- Notebook heatmap: text-based representation of the stance heatmap (accessible in plain text)

The output file is a single .md file suitable for pasting into a research memo, uploading to GitHub, or feeding into another AI session for meta-analysis.

8.3 Citation Copy

Right-clicking any Point in Pane 2 or Pane 3 shows ‘Copy Citation’. The copied text includes: source title, author(s), date, publication, URL, and the Point text as a quotation block. Format is configurable: Chicago | APA | BibTeX | Plain.

8.4 Analysis JSON Export

For researchers building downstream tools (including the POViewer pipeline scripts), File → Export → Analysis JSON exports the merged AI + human annotation data for one or all sources as raw JSON. This is the data format consumed by batch_summarize.py and detect_conflicts.py.

9. Workstream G — Long Document Handling

Phase 1 noted that documents over ~100,000 tokens would hit context limits and require chunking. Phase 2 implements a chunked analysis strategy that handles arbitrarily long documents without losing cross-document coherence in the mapping.

9.1 Chunking Strategy

Documents longer than 80,000 tokens (approx. 60,000 words) are automatically chunked before analysis. Chunking is semantic, not mechanical: the AI identifies natural section boundaries (headings, chapter breaks, logical transitions) and splits at those points rather than at fixed token counts.

Stage	Chunking approach
Stage 1 (Segmentation)	Each chunk is segmented independently. Point IDs are prefixed with the chunk index to ensure uniqueness: p-03-014 = chunk 3, point 14.
Stage 2 (Mapping)	Each chunk's Points are mapped independently. The taxonomy context travels with every chunk call.
Cross-chunk coherence	After all chunks complete, a lightweight reconciliation pass checks for Points that span a chunk boundary (split claims) and merges them.

9.2 Progress UI for Long Documents

For documents with more than 5 chunks, a document-level progress panel replaces the simple spinner:

Element	Content
Chunk map	A horizontal strip divided into N segments, each shading in as its chunk completes.
ETA estimate	Based on average seconds per chunk so far: '~4 min remaining'
Partial display	Pane 2 shows highlights for completed chunks immediately — the document is usable before analysis finishes.
Pause / resume	The user can pause chunked analysis between chunks and resume later. The job persists in the queue file.

9.3 Virtual Scroll Rendering

Pane 2 uses virtual scrolling for all long documents regardless of analysis state. Only the pages/paragraphs within the current viewport plus a 2-screen buffer are rendered to the DOM. This keeps the UI responsive for documents of any length without pagination.

10. Workstream H — Public Beta Readiness

Phase 2 concludes with a public beta release to a selected group of external researchers beyond BKC. This requires UX polish, onboarding, error handling, accessibility, and basic telemetry — none of which were needed for the internal prototype.

10.1 Onboarding Flow

First launch after installation shows a 4-step setup wizard:

1. Welcome: brief explanation of POViewer and the AI Triad project. Link to BKC website.
2. AI Provider: prompt for API key (with direct links to get a Gemini or Claude key). Test button. Explain that the key is stored securely and never leaves the device.
3. Load Taxonomy: file picker to load POV taxonomy JSON files. A 'Use AI Triad taxonomy' button auto-downloads the four canonical files from the BKC GitHub repository.
4. Create First Notebook: name field, optional description, 'Add your first source' prompt.

10.2 Error Handling Standards

Every error state must have three things: a plain-English explanation of what went wrong, a specific action the user can take to fix it, and a way to copy technical details for bug reporting.

Error scenario	Message shown to user
AI API key missing	'No API key configured. Go to Settings → AI Provider to add your key.'
API key invalid (401)	'Your API key was rejected. Check that it is correct and active in your provider dashboard.'
Rate limit (429)	'Analysis paused — rate limit reached. Will retry automatically in 30 seconds.'
Document fetch failed	'Could not retrieve this URL. The site may block automated access. Try downloading the PDF manually and adding it as a file.'
PDF extraction failed	'Could not extract text from this PDF (it may be image-based). Try installing pdftotext: [link].'
Taxonomy file not found	'Taxonomy file missing at saved path. Click here to re-locate it.'

10.3 Telemetry (Opt-in)

POViewer collects anonymous usage telemetry only with explicit opt-in consent during onboarding. Telemetry helps the BKC team prioritise features and detect crashes. Data collected:

- App launch count and session duration

- Feature usage counts (which views/tabs are used, not content)
- Error types and frequencies (no stack traces containing user data)
- Document counts and analysis completion rates (no document content or URLs)

Telemetry is sent to a BKC-controlled endpoint. It is never sold or shared with third parties. It can be disabled in Settings at any time.

10.4 Accessibility

Requirement	Implementation
Keyboard navigation	Full tab order through all three panes. Highlights navigable with arrow keys. All actions available without mouse.
Screen reader support	All highlights have aria-label with Point text and mapping count. Pane 3 detail card uses semantic HTML (dl/dt/dd for mappings).
Colour-blind mode	Settings toggle replaces POV colours with high-contrast patterns: Accelerationist = diagonal stripes, Safetyist = dots, Skeptic = crosshatch. Icons remain.
Font size	Global font scale control in Settings (80% – 150%).
WCAG target	WCAG 2.1 AA for all UI chrome and Pane 3. Pane 2 document text inherits source formatting.

11. Complete Feature Matrix: Phase 1 + Phase 2

Feature	Phase 1	Phase 2
Three-pane layout, resizable	✓ Complete	—
Notebook create / switch	✓ Complete	—
Add source: URL	✓ Complete	—
Add source: PDF	✓ Scaffolded	Native PDF rendering (Section 4)
Source select / deselect	✓ Complete	—
Markdown snapshot rendering in Pane 2	✓ Complete	—
PDF rendering in Pane 2	✗ Deferred	✓ PDF.js viewer + highlight overlay (Section 4)
Point highlight: single-POV	✓ Complete	—
Point highlight: multi-POV badge	✓ Complete	—
Agree / contradict icon overlay	✓ Complete	—
Click highlight → Pane 3 detail	✓ Complete	—
POV filter toggles	✓ Complete	—
Pane 3: per-camp mapping blocks	✓ Complete	—
Pane 3: cross-cutting collision alert	✓ Complete	—
Pane 3: unmapped point display	✓ Basic	Taxonomy Gap View with clustering (Section 7)
Taxonomy file loading from disk	✓ Complete	—
Taxonomy session persistence	✓ Complete	—
Live AI analysis: segmentation	✗ Deferred	✓ Two-stage pipeline (Section 3)
Live AI analysis: mapping	✗ Deferred	✓ Two-stage pipeline (Section 3)
Analysis status model	✗ Deferred	✓ 9-state model (Section 3.4)
Chunked analysis for long docs	✗ Deferred	✓ Semantic chunking (Section 9)
Virtual scroll rendering	✗ Deferred	✓ (Section 9.3)
User annotation: override mapping	✗ Deferred	✓ (Section 5)
User annotation: add / remove mapping	✗ Deferred	✓ (Section 5)
User annotation: research notes	✗ Deferred	✓ (Section 5)

Feature	Phase 1	Phase 2
Cross-source Node View	✗ Deferred	✓ Pane 3 Node View mode (Section 6)
Notebook-level stance heatmap	✗ Deferred	✓ Overview screen (Section 6.3)
Taxonomy Gap View with clustering	✗ Deferred	✓ Pane 3 Gaps tab (Section 7)
Taxonomy proposal flow	✗ Deferred	✓ Propose Node side panel (Section 7.2)
Notebook bundle export (.poviewer)	✗ Deferred	✓ (Section 8.1)
Markdown report export	✗ Deferred	✓ (Section 8.2)
Citation copy	✗ Deferred	✓ (Section 8.3)
Analysis JSON export	✗ Deferred	✓ (Section 8.4)
Onboarding wizard	✗ Deferred	✓ (Section 10.1)
Error handling standards	✗ Deferred	✓ (Section 10.2)
Accessibility (WCAG 2.1 AA)	✗ Deferred	✓ (Section 10.4)
Colour-blind mode	✗ Deferred	✓ (Section 10.4)
Web-hosted version	✗ Deferred	✗ Phase 3
Multi-user collaboration (live)	✗ Deferred	✗ Phase 3

12. Phase 3 Horizon

Phase 3 is not specified here but is noted for planning purposes. The following capabilities are explicitly out of scope for Phase 2:

Phase 3 capability	Dependency on Phase 2
Web-hosted version	Requires backend service, auth, storage. Phase 2 desktop app must first prove the full feature set is stable before investing in web infrastructure.
Live multi-user collaboration	Requires a shared annotation sync layer (CRDT or operational transform). Phase 2 establishes the annotation data model that Phase 3 will sync.
Public taxonomy contribution	A workflow for external researchers to submit taxonomy proposals to the BKC canonical repository. Requires the Phase 2 Proposal flow as its foundation.
Citation network view	Visualise how documents in the Notebook cite each other and which taxonomy nodes are shared. Requires the Phase 2 cross-source data layer.
Automated taxonomy versioning via GitHub	The Phase 1 GitHub Actions integration re-runs batch_summarize.py on taxonomy changes. Phase 3 connects POViewer directly to the repo so taxonomy edits in the app create PRs.